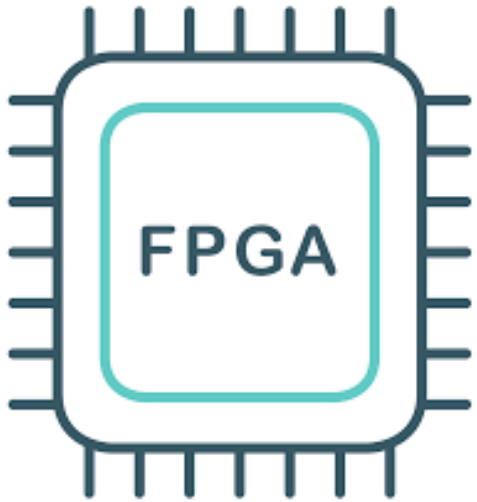
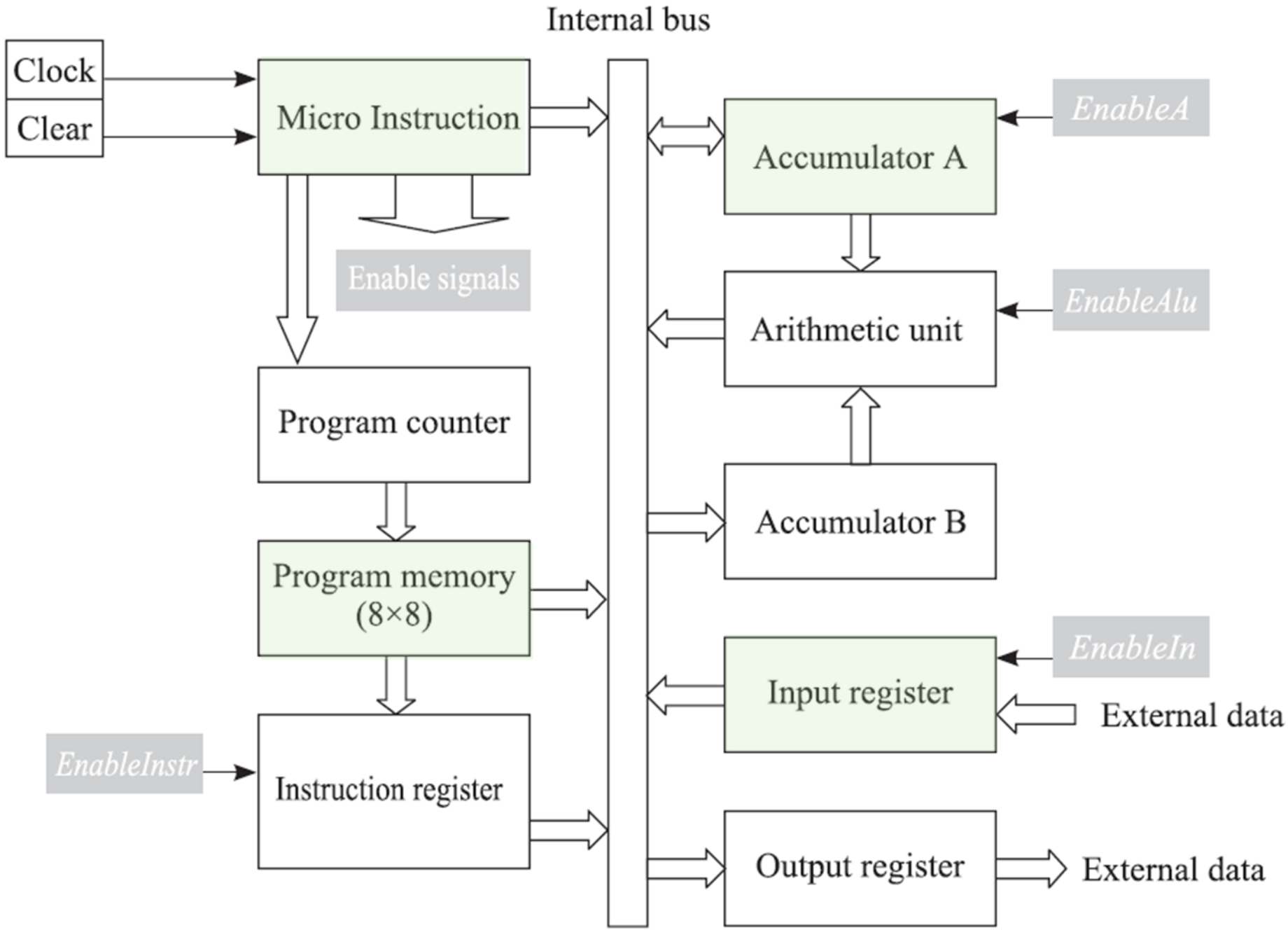
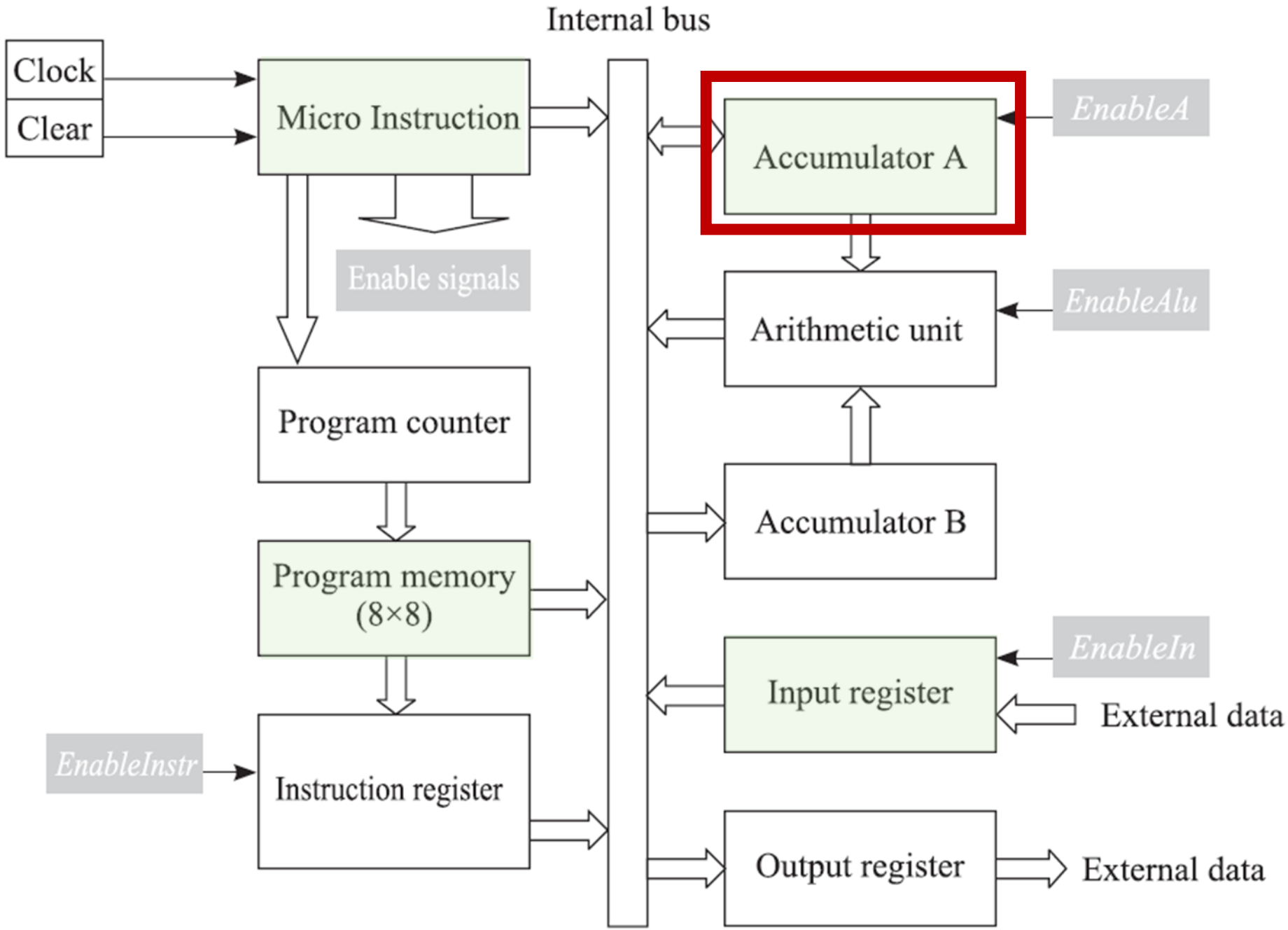
Very Simple Microprocessor (Verilog code)

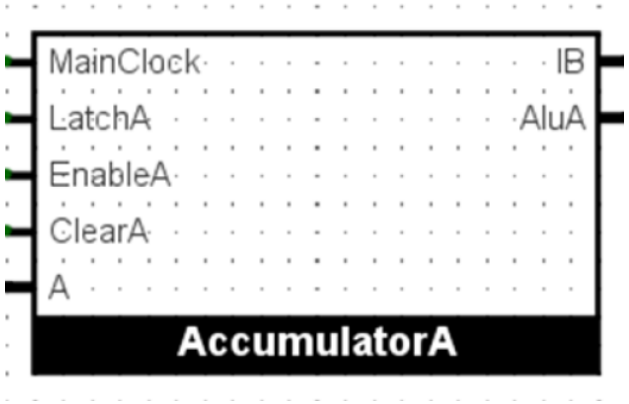


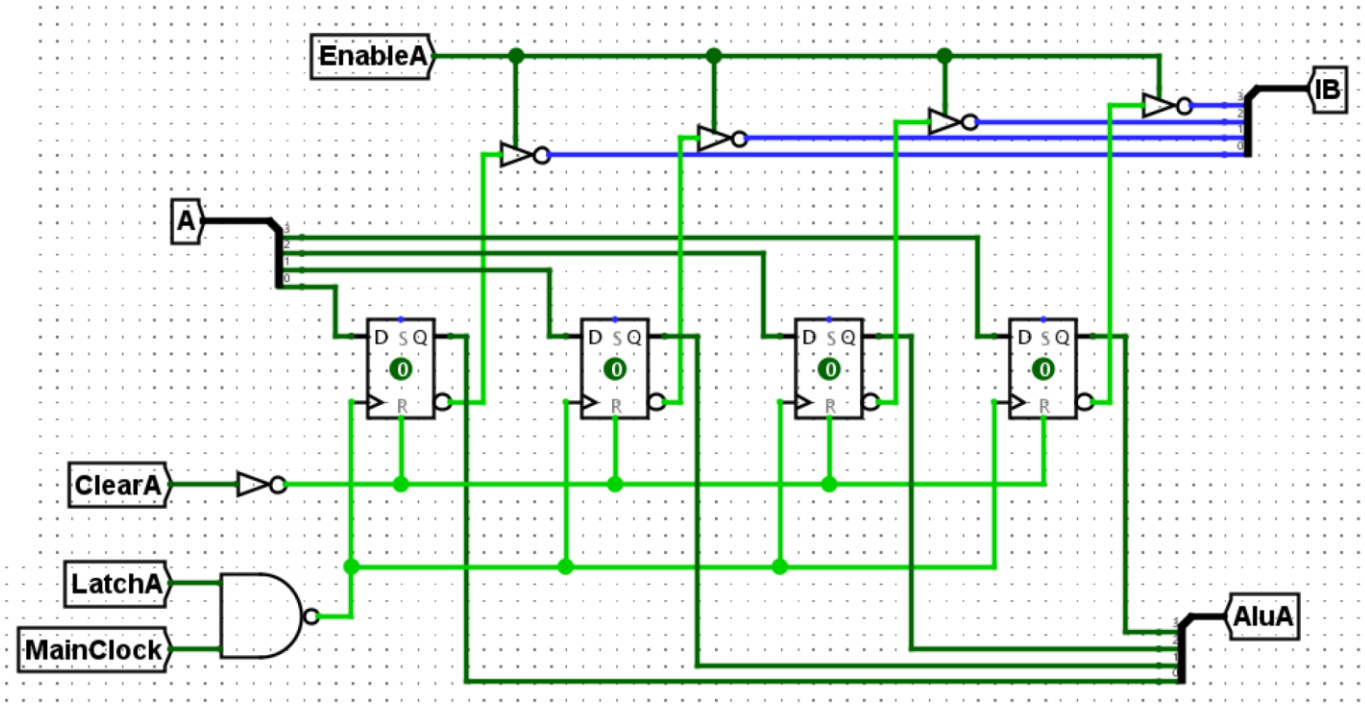

Internal bus
Clock
Clear
Micro Instruction
Enable signals
Program counter
Program memory
(8×8)
EnableInstr
Instruction register
EnableA
Accumulator A
Arithmetic unit
EnableAlu
Accumulator B
EnableIn
Input register
External data
Output register
External data


Internal bus
Clock
Clear
Micro Instruction
Enable signals
Program counter
Program memory
(8×8)
EnableInstr
Instruction register
Accumulator A
EnableA
Arithmetic unit
EnableAlu
Accumulator B
Input register
EnableIn
External data
Output register
External data

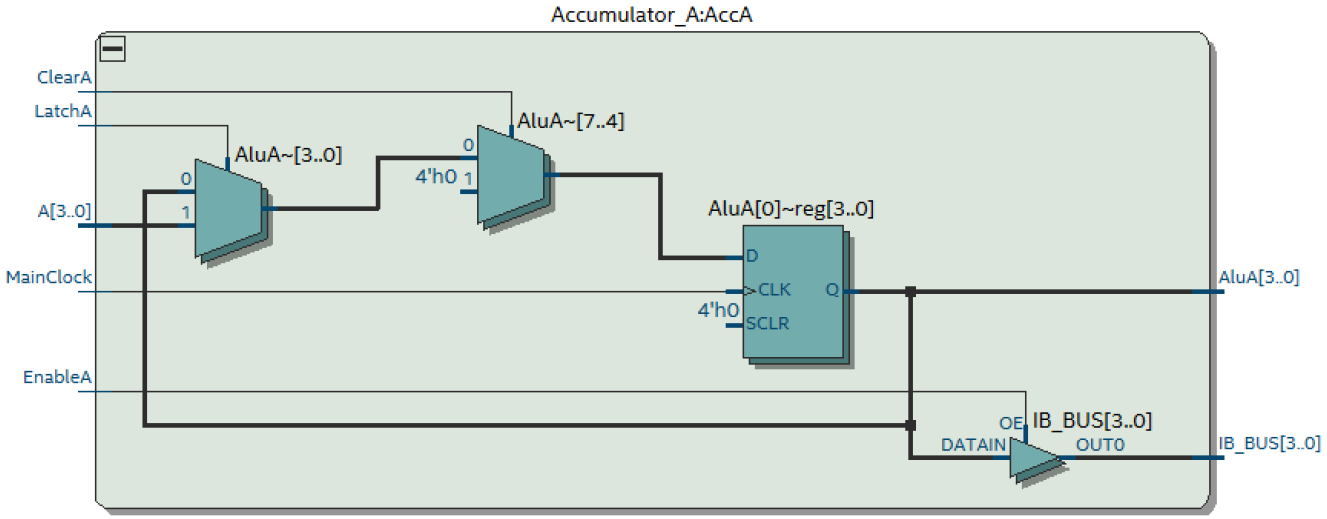
Accumulator A – Code

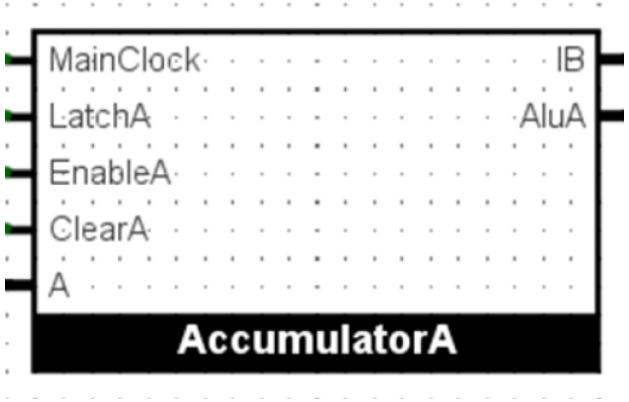
```
1
2   `default_nettype none
3
4   module Accumulator_A #
5   (
6       parameter N = 4
7   )
8   (
9       input MainClock,
10      input ClearA,
11      input LatchA,
12      input EnableA,
13      input [N-1:0] A,
14      output [N-1:0] IB_BUS,
15      output reg [N-1:0] AluA
16  );
17
18      // Sequential Logic
19      always @(posedge MainClock) begin
20          if(ClearA)
21              AluA <= {N{1'b0}};
22          else begin
23              if(LatchA)
24                  AluA <= A;
25          end
26      end
27
28      // Combinational Logic
29      assign IB_BUS = (EnableA) ? AluA : {N{1'bz}};
30
31  endmodule
32
33  `default_nettype wire
```

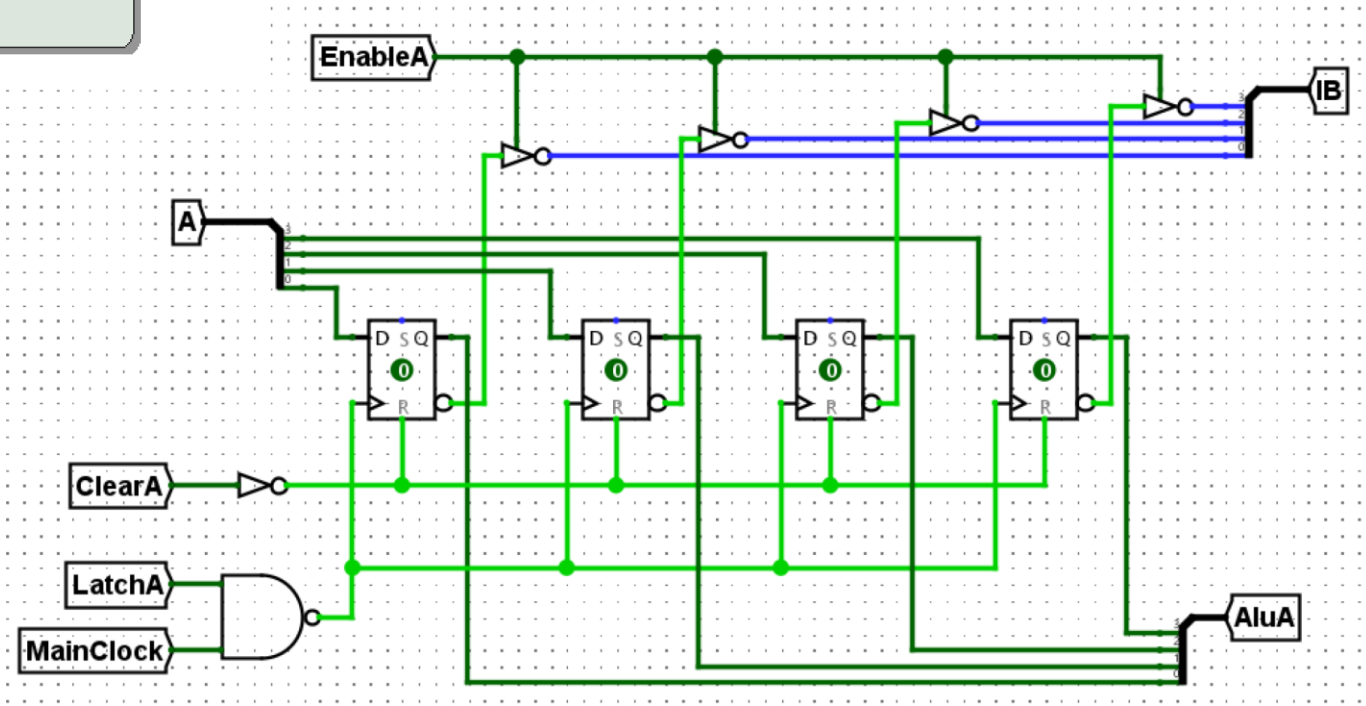




Accumulator A – RTL Viewer

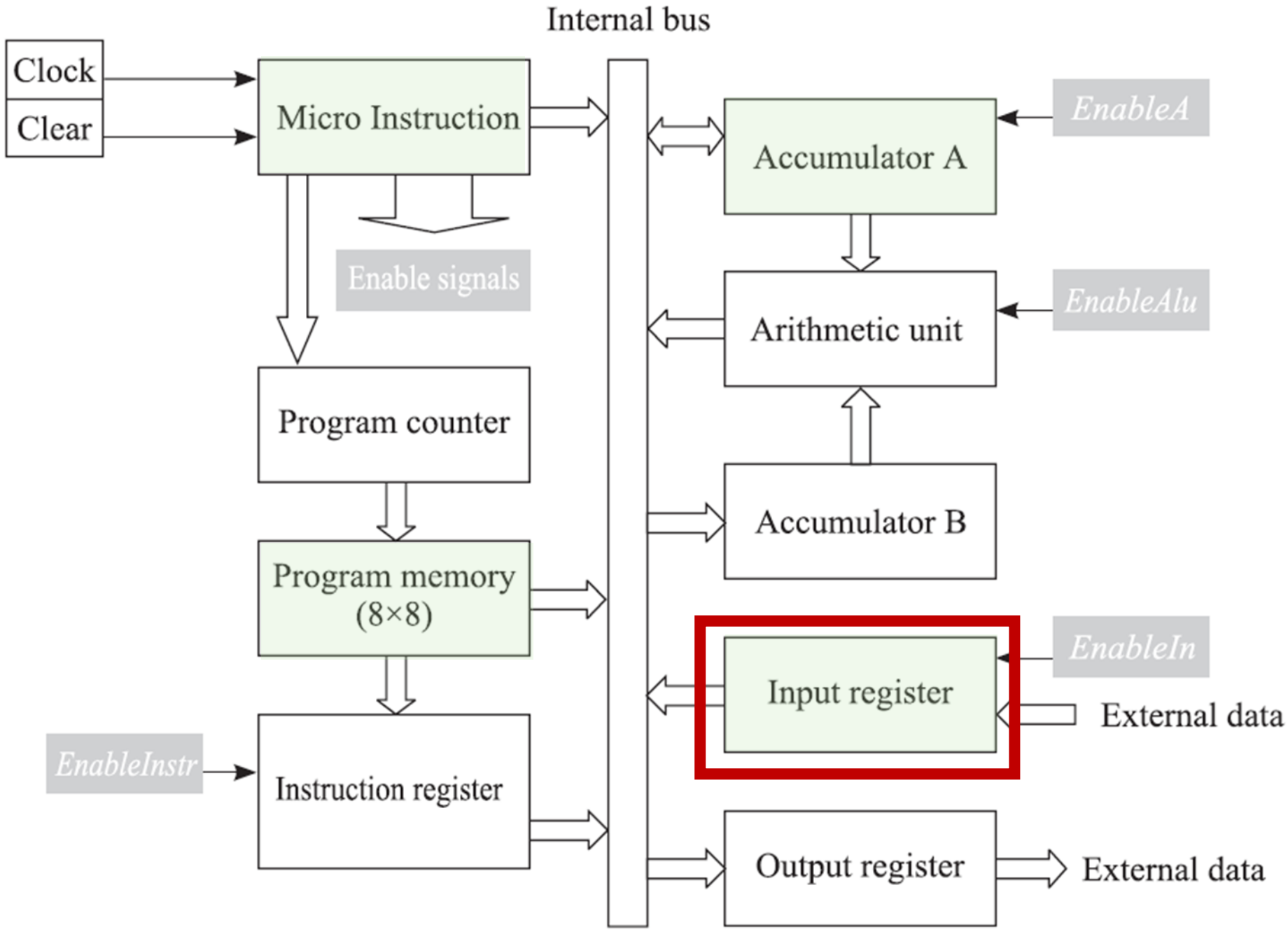






Accumulator A – Test

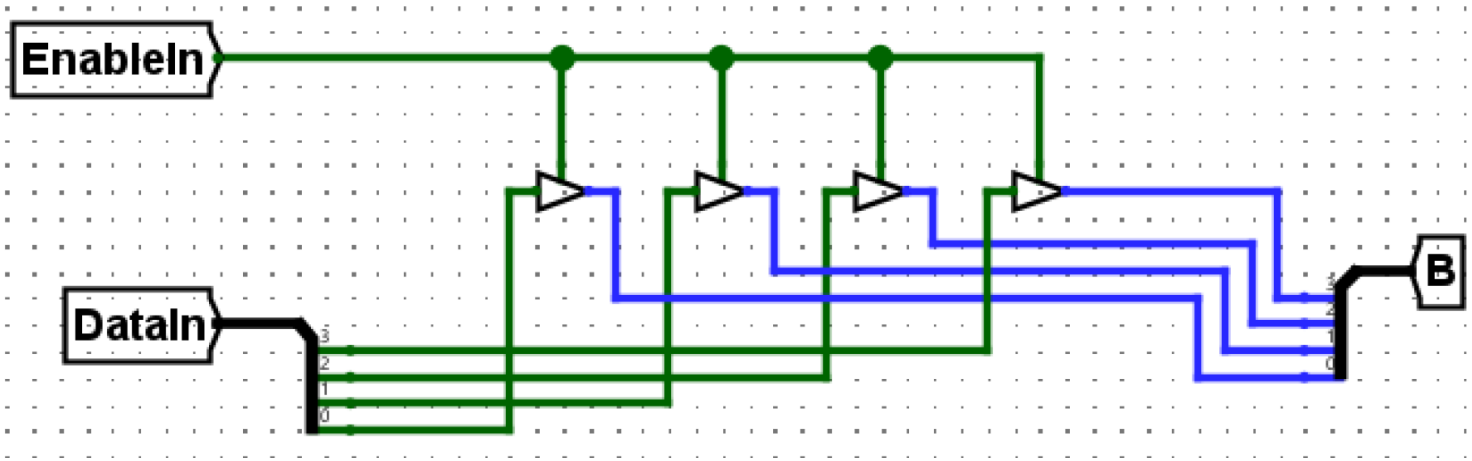
```
// Accumulator A (default 4bits)
Accumulator_A AccA(
    .MainClock(w_clock),
    .ClearA(w_reset),
    .LatchA(w_LatchA),        // FSM CONTROL
    .EnableA(w_EnableA),      // FSM CONTROL
    .A(w_IB_BUS),
    .IB_BUS(w_IB_BUS),
    .AluA(w_AluA)
);
```


Internal bus
Clock
Clear
Micro Instruction
Enable signals
Program counter
Program memory
(8×8)
EnableInstr
Instruction register
Accumulator A
EnableA
Arithmetic unit
EnableAlu
Accumulator B
Input register
EnableIn
External data
Output register
External data

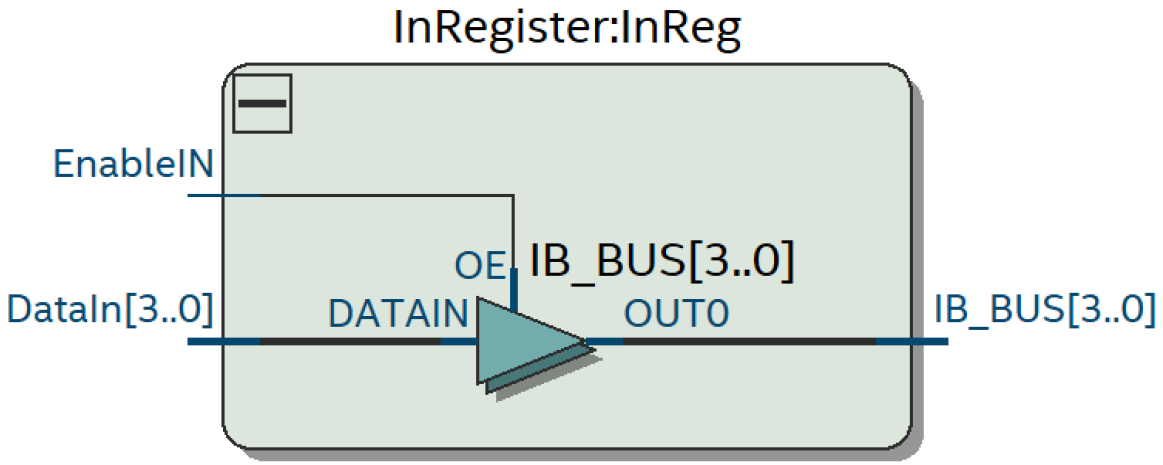
InRegister – Code

```
1
2   `default_nettype none
3
4   module InRegister #
5   (
6       parameter N = 4
7   )
8   (
9       input EnableIN,
10      input [N-1:0] DataIn,
11      output [N-1:0] IB_BUS
12  );
13
14      assign IB_BUS = (EnableIN) ? DataIn : {N{1'bz}};
15
16  endmodule
17
18  `default_nettype wire
```

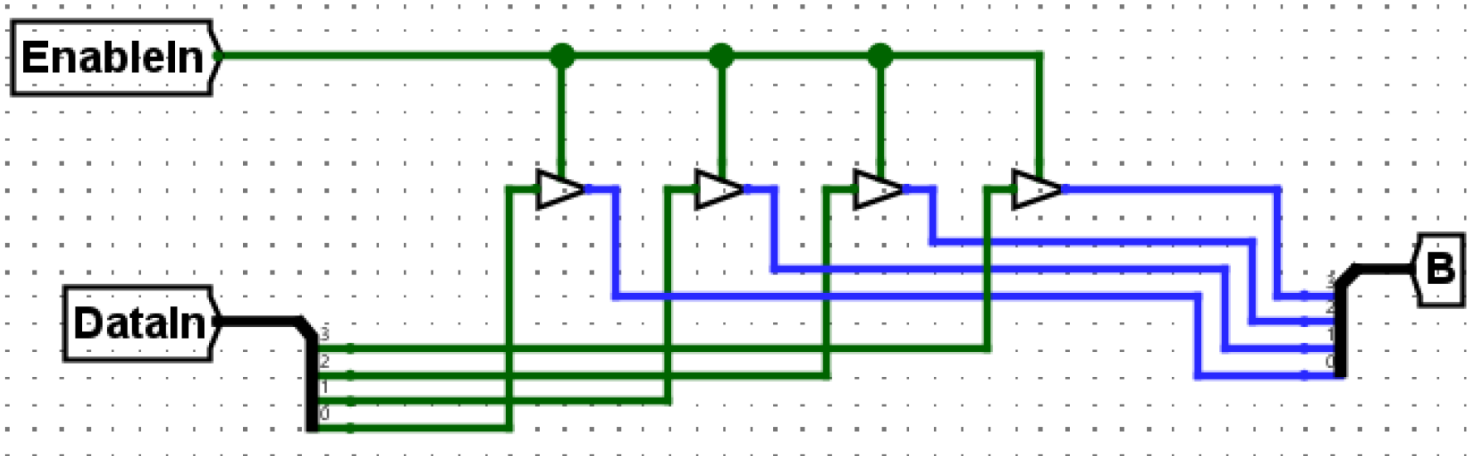




InRegister – RTL Viewer

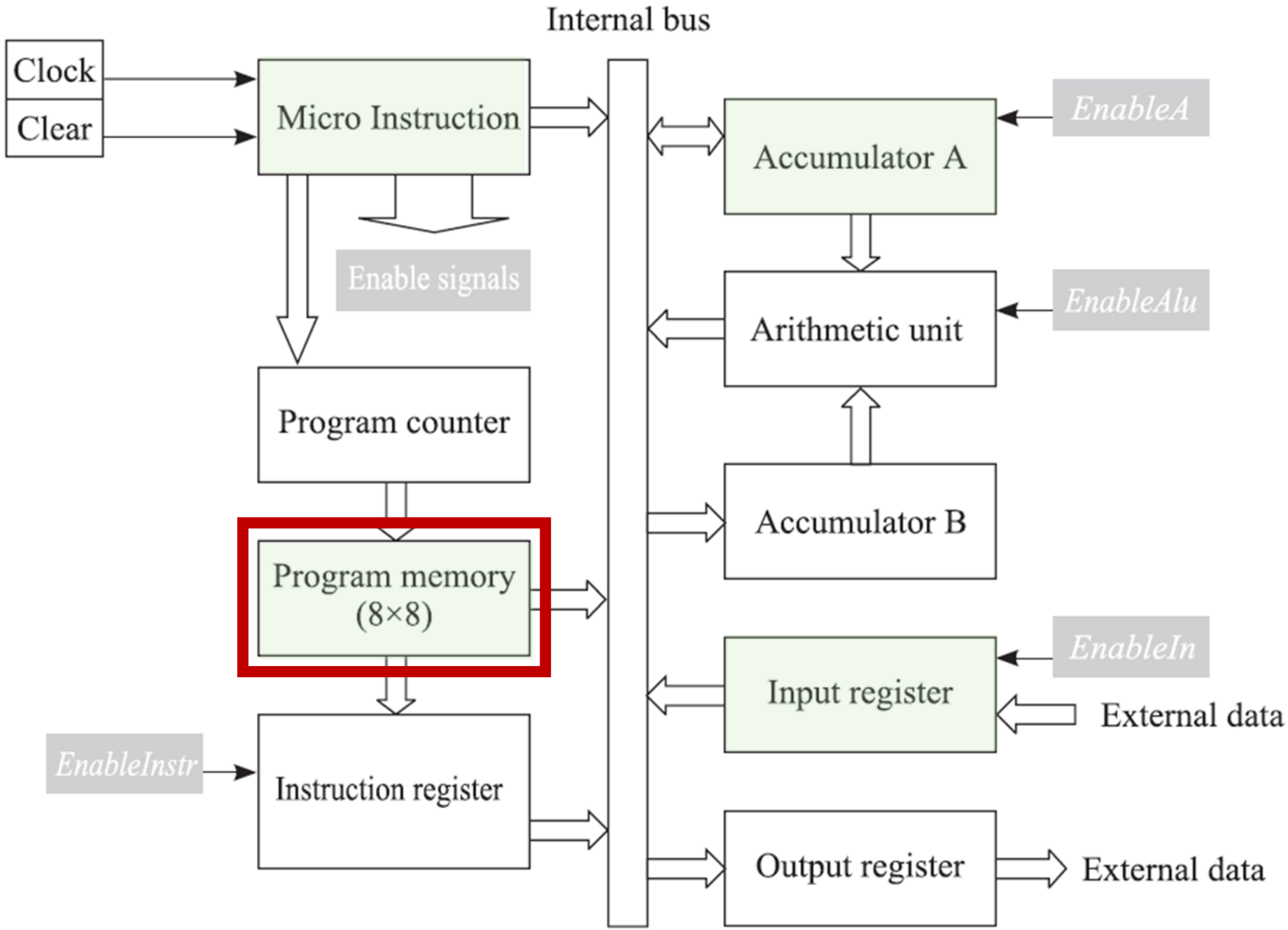






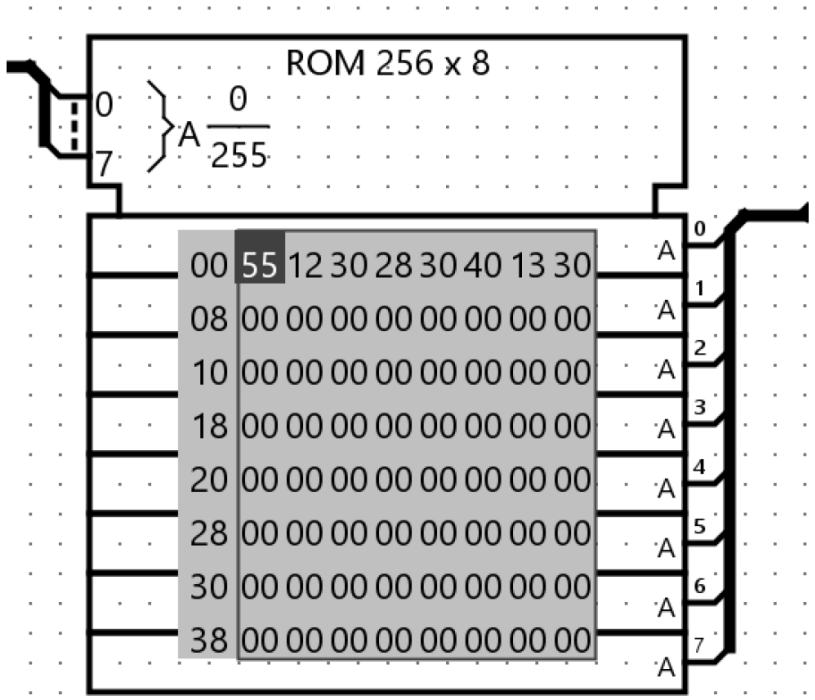
InRegister – Test

```
// Input Register (default 4bits)
InRegister InReg(
    .EnableIN(w_EnableIN),  // FSM CONTROL
    .DataIn(w_user_input),
    .IB_BUS(w_IB_BUS)
);
```

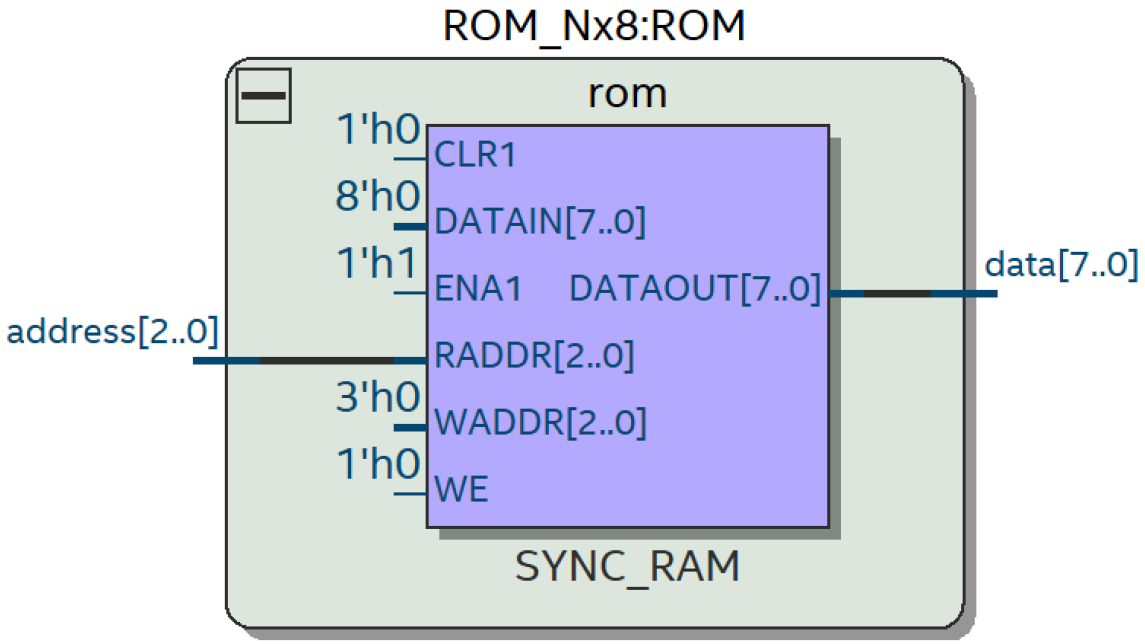

Internal bus
Clock
Clear
Micro Instruction
Enable signals
Program counter
Program memory
(8×8)
EnableInstr
Instruction register
EnableA
Accumulator A
Arithmetic unit
EnableAlu
Accumulator B
EnableIn
Input register
External data
Output register
External data

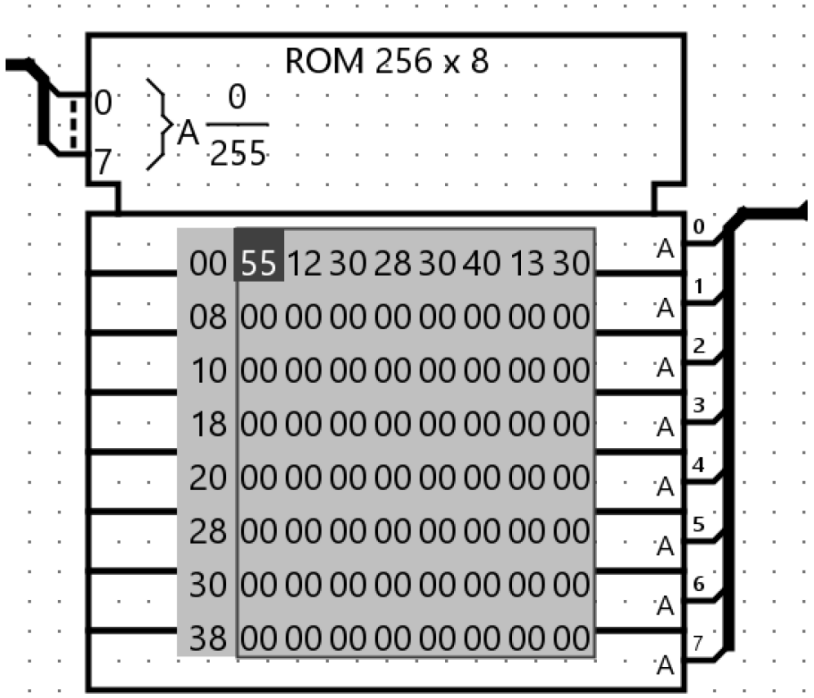
Memory – Code

```
1
2     `default_nettype none
3
4     module ROM_Nx8 #
5     (
6        parameter N = 8
7     )
8     (
9        input [$clog2(N)-1:0] address,
10       output reg [7:0] data
11    );
12
13       (* ramstyle = "logic" *) reg [7:0] rom [0:N-1];
14
15       // Initialize memory -> your asm instructions
16       initial begin
17          rom[0] = 8'h55;
18          rom[1] = 8'h12;
19          rom[2] = 8'h30;
20          rom[3] = 8'h28;
21          rom[4] = 8'h30;
22          rom[5] = 8'h40;
23          rom[6] = 8'h13;
24          rom[7] = 8'h30;
25          // maximum 16 instructions
26       end
27
28       // Combinational Logic (async)
29       always @(*) begin
30          data = rom[address];
31       end
32
33    endmodule
34
35    `default_nettype wire
36
```



Memory – RTL Viewer





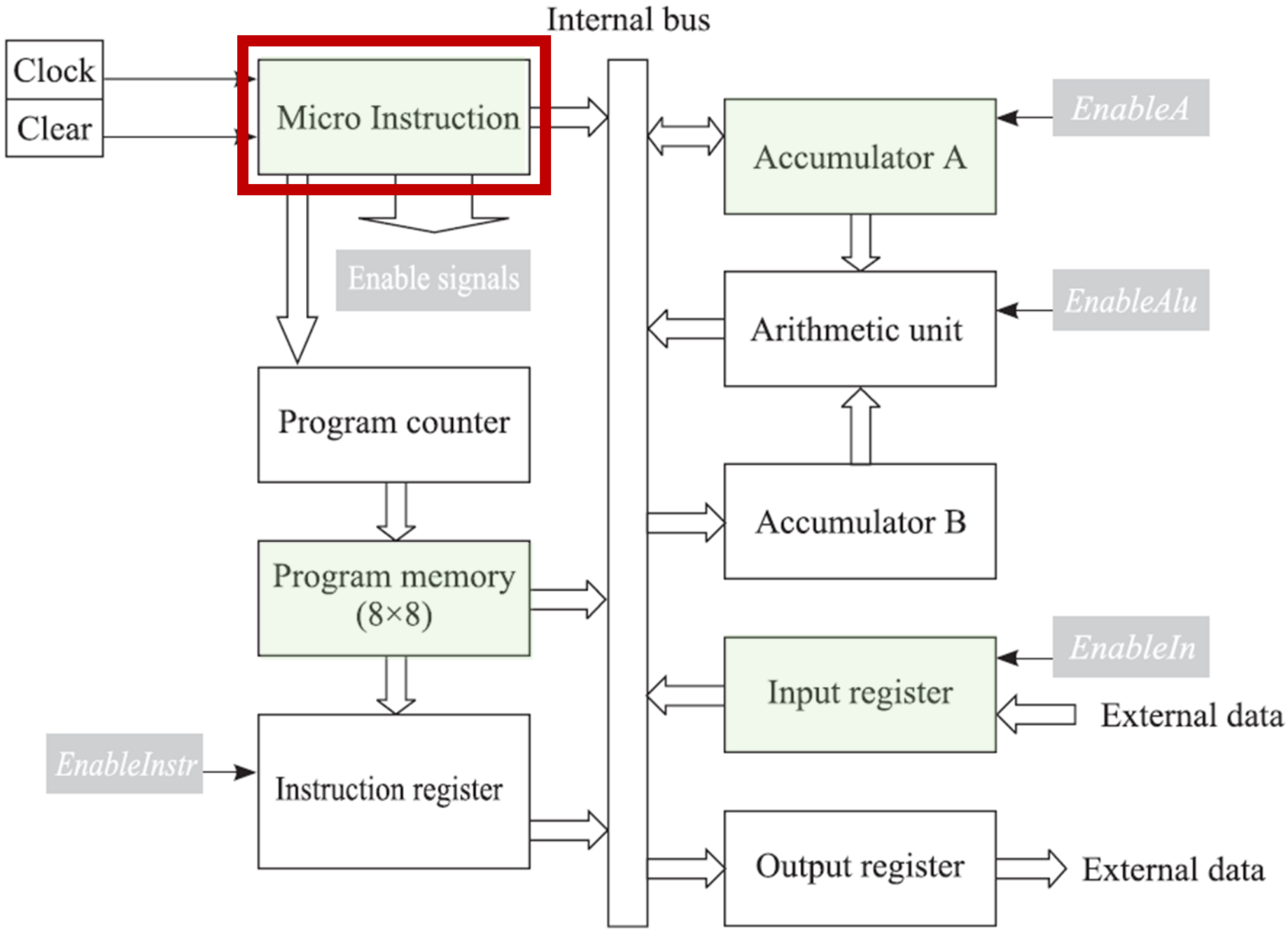
Memory – Test

```
// Memory ROM 8x8

wire [7:0] w_rom_data;

ROM_Nx8 ROM (
    .address(w_counter[2:0]),
    .data(w_rom_data)
);

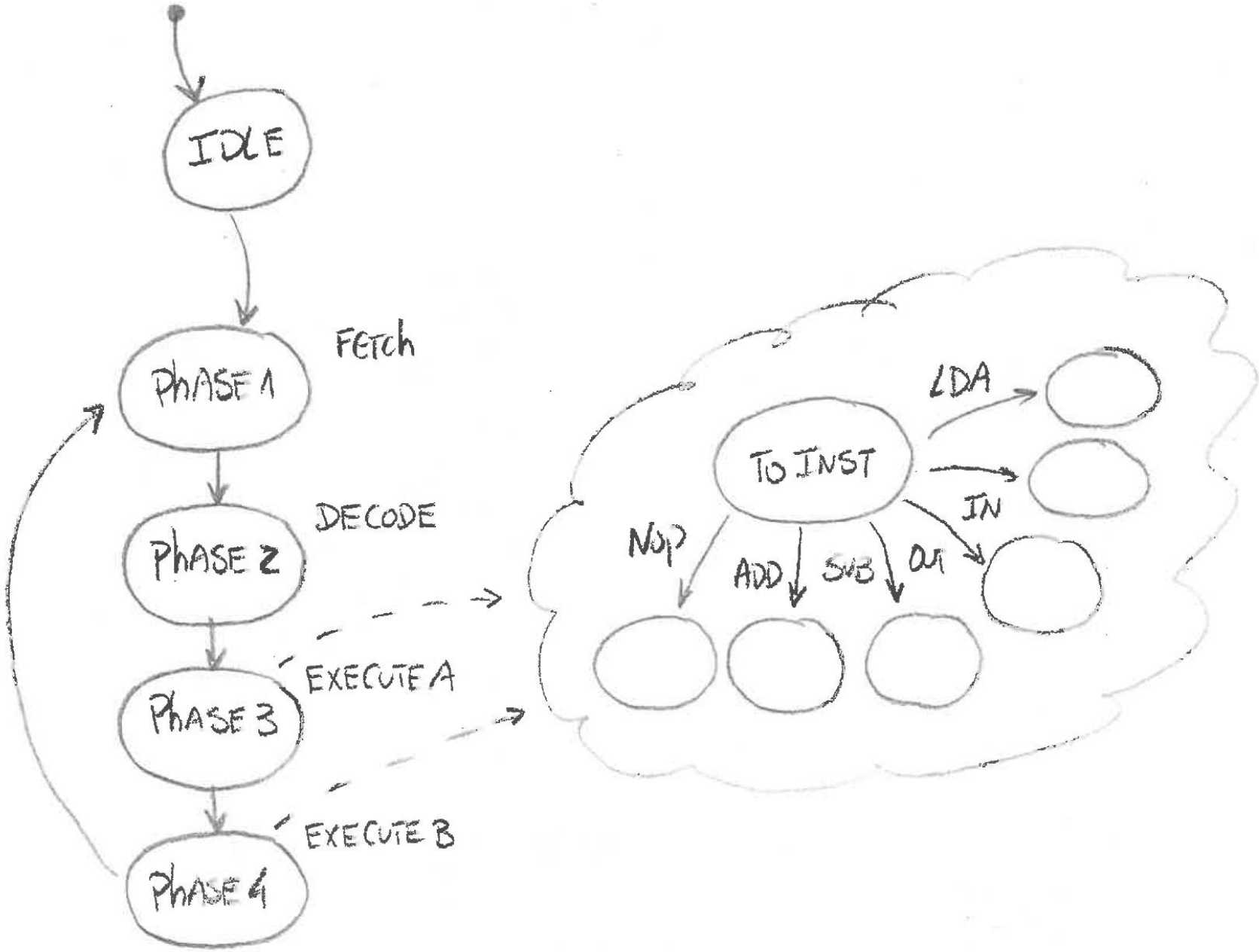
assign {w_instruction, w_data} = w_rom_data;
```


Internal bus
Clock
Clear
Micro Instruction
Enable signals
Program counter
Program memory
(8×8)
EnableInstr
Instruction register
EnableA
Accumulator A
Arithmetic unit
EnableAlu
Accumulator B
EnableIn
Input register
External data
Output register
External data

		Phase 1	Phase 2	Phase 3	Phase 4
		0001	0010	0100	1000
(NOP): 0000	ProgCount	0	1	0	0
	EnableOut	0	0	0	0
	LoadInstr	1	0	0	0
	LatchB	0	0	0	0
	LatchA	0	0	0	0
	EnableInstr	0	1	0	0
	EnableIn	0	0	0	0
	EnableAlu	0	0	0	0
	EnableA	0	0	0	0
	AddSub	0	0	0	0
(ADD): 0001	ProgCount	0	1	0	0
	EnableOut	0	0	0	0
	LoadInstr	1	0	0	0
	LatchB	0	0	1	0
	LatchA	0	0	0	1
	EnableInstr	0	1	1	0
	EnableIn	0	0	0	0
	EnableAlu	0	0	0	1
	EnableA	0	0	0	0
	AddSub	0	0	0	0

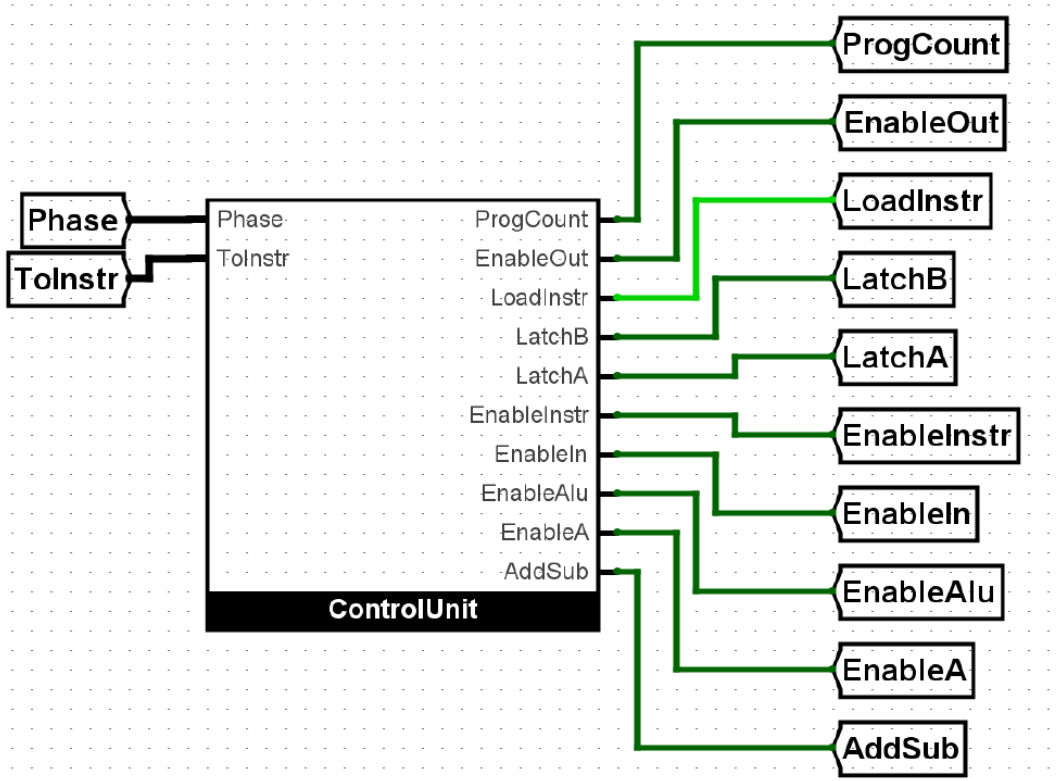
		Phase 1	Phase 2	Phase 3	Phase 4
		0001	0010	0100	1000
(SUB): 0010	ProgCount	0	1	0	0
	EnableOut	0	0	0	0
	LoadInstr	1	0	0	0
	LatchB	0	0	1	0
	LatchA	0	0	0	1
	EnableInstr	0	1	1	0
	EnableIn	0	0	0	0
	EnableAlu	0	0	0	1
	EnableA	0	0	0	0
	AddSub	0	0	0	1
(In): 0100	ProgCount	0	1	0	0
	EnableOut	0	0	0	0
	LoadInstr	1	0	0	0
	LatchB	0	0	0	0
	LatchA	0	0	1	0
	EnableInstr	0	1	0	0
	EnableIn	0	0	1	0
	EnableAlu	0	0	0	0
	EnableA	0	0	0	0
	AddSub	0	0	0	0

		Phase 1	Phase 2	Phase 3	Phase 4
		0001	0010	0100	1000
(Out): 0011	ProgCount	0	1	0	0
	EnableOut	0	0	1	0
	LoadInstr	1	0	0	0
	LatchB	0	0	0	0
	LatchA	0	0	0	0
	EnableInstr	0	1	0	0
	EnableIn	0	0	0	0
	EnableAlu	0	0	0	0
	EnableA	0	0	1	0
	AddSub	0	0	0	0
(LDA): 0101	ProgCount	0	1	0	0
	EnableOut	0	0	0	0
	LoadInstr	1	0	0	0
	LatchB	0	0	0	0
	LatchA	0	0	1	0
	EnableInstr	0	1	1	0
	EnableIn	0	0	0	0
	EnableAlu	0	0	0	0
	EnableA	0	0	0	0
	AddSub	0	0	0	0


IDLE
PHASE 1
FETCH
PHASE 2
DECODE
PHASE 3
EXECUTE A
PHASE 4
EXECUTE B
TO INST
LDA
IN
NOP
ADD
SUB
OUT

FSM – Code

```
1
2     `default_nettype none
3
4     module FSM_MicroInstr #
5     (
6         parameter N = 4
7     )
8     (
9         input clk,
10        input reset,
11        input [N-1:0] IB_BUS,
12
13        output reg LatchA,
14        output reg EnableA,
15        output reg LatchB,
16        output reg EnableALU,
17        output reg AddSub,
18        output reg EnableIN,
19        output reg EnableOut,
20        output reg LoadInstr,
21        output reg EnableInstr,
22        input [N-1:0]  ToInstr,
23        output reg EnableCount
24    );
25
```



FSM – Code

```
26      reg [2:0] state, next_state;
27
28      // States
29      localparam [2:0] IDLE = 3'd0, PHASE_1 = 3'd1, PHASE_2 = 3'd2, PHASE_3 = 3'd3, PHASE_4 = 3'd4;
30
31      // 1. State Register (sequential)
32      always @(posedge clk) begin
33          if(reset)
34              state <= IDLE;
35          else
36              state <= next_state;
37      end
38
```

FSM – Code

```
38
39      // 2. Next-State Logic (combinational)
40      always @(*) begin
41
42          next_state = state; // default: hold state
43
44          case(state)
45
46              IDLE:
47                  begin
48                      // Move to FETCH
49                      next_state = PHASE_1;
50                  end
51
52              // FETCH Instruction
53              PHASE_1:
54                  begin
55                      // Move to DECODE
56                      next_state = PHASE_2;
57                  end
58
59              // DECODE Instruction
60              PHASE_2:
61                  begin
62                      // Move to EXECUTE A
63                      next_state = PHASE_3;
64                  end
65
66              // EXECUTE A Instruction
67              PHASE_3:
68                  begin
69                      // Move to EXECUTE B
70                      next_state = PHASE_4;
71                  end
72
73              // EXECUTE B Instruction
74              PHASE_4:
75                  begin
76                      // Move to FETCH
77                      next_state = PHASE_1;
78                  end
79
80              default: ;// None
81
82          endcase
83      end
84
```

FSM – Code

```
85      // 3. Output Logic (combinational)
86
87      always @(*) begin
88
89          // default all signals are zero
90          LoadInstr   = 1'b0;
91          EnableInstr = 1'b0;
92          LatchB      = 1'b0;
93          LatchA      = 1'b0;
94          EnableALU   = 1'b0;
95          EnableCount = 1'b0;
96          AddSub      = 1'b0;
97          EnableIN    = 1'b0;
98          EnableA     = 1'b0;
99          EnableOut   = 1'b0;
100
101         case(state)
102
103             IDLE:
104                 begin
105                     // Initialize things if needed
106                 end
107
108             // FETCH Instruction
109             PHASE_1:
110                 begin
111                     // same sequence for all asm instructions
112                     LoadInstr = 1'b1;
113                 end
114
115             // DECODE Instruction
116             PHASE_2:
117                 begin
118                     // same sequence for all asm instructions
119                     EnableCount = 1'b1;
120                     EnableInstr = 1'b1;
121                 end
```

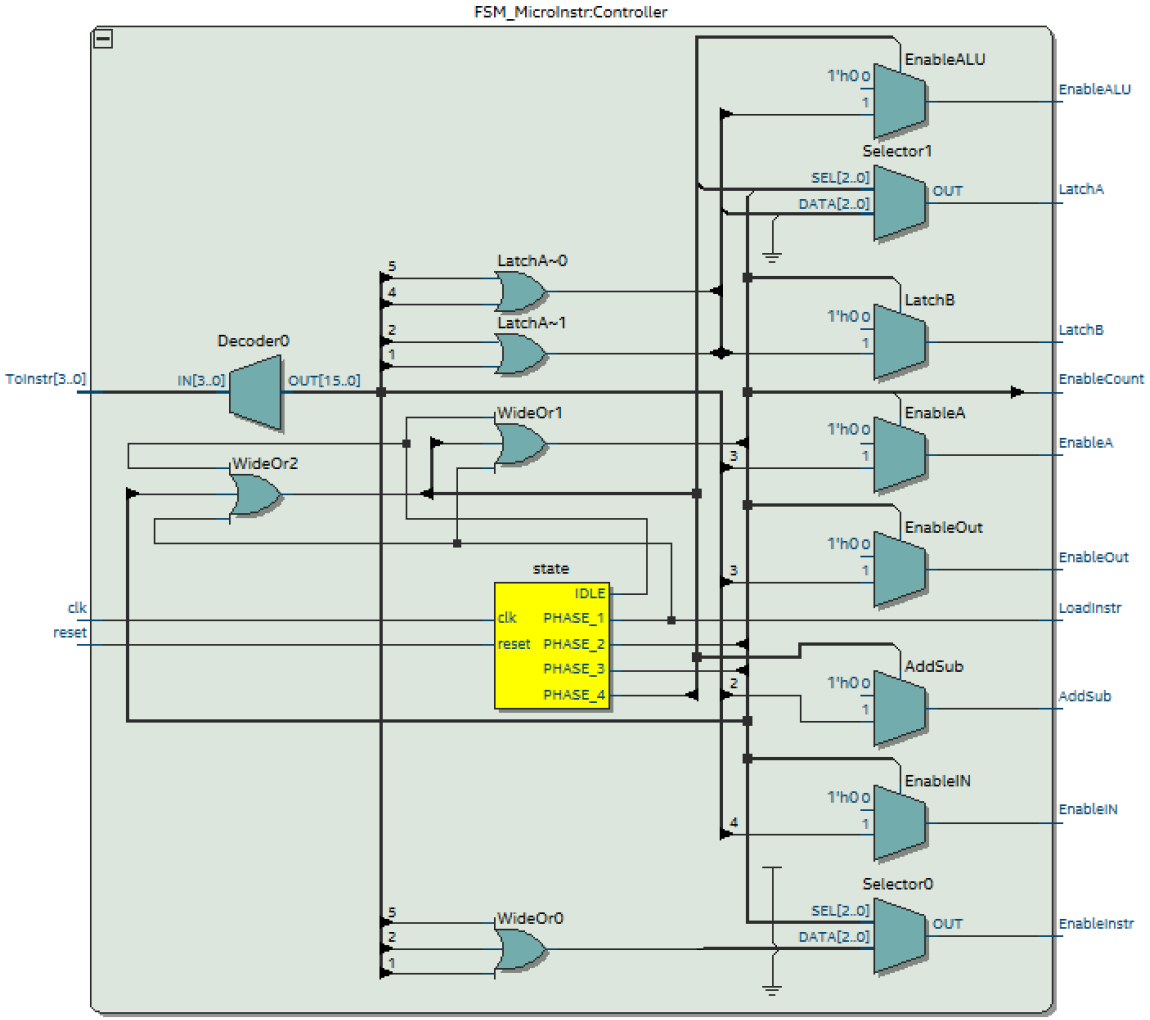
FSM – Code

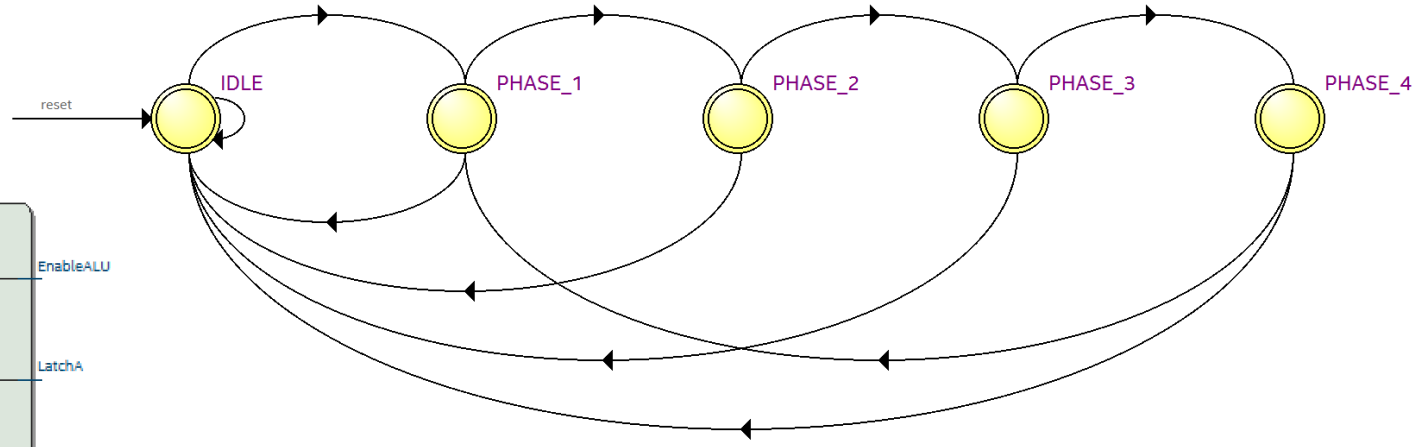
```
122
123         // EXECUTE Instruction
124         PHASE_3:
125            begin
126
127               case(ToInstr)
128
129                  4'b0000: // NOP
130                     begin
131                        // do nothing
132                     end
133
134                  4'b0001: // ADD
135                     begin
136                        LatchB      = 1'b1;
137                        EnableInstr = 1'b1;
138                     end
139
140                  4'b0010: // SUB
141                     begin
142                        LatchB      = 1'b1;
143                        EnableInstr = 1'b1;
144                     end
145
146                  4'b0011: // OUT
147                     begin
148                        EnableOut   = 1'b1;
149                        EnableA     = 1'b1;
150                     end
151
152                  4'b0100: // IN
153                     begin
154                        LatchA   = 1'b1;
155                        EnableIN = 1'b1;
156                     end
157
158                  4'b0101: // LDA
159                     begin
160                        LatchA      = 1'b1;
161                        EnableInstr = 1'b1;
162                     end
163
164               endcase
165
166            end
```

FSM – Code

```
167
168         // EXECUTE Instruction
169         PHASE_4:
170            begin
171
172               case(ToInstr)
173
174                  4'b0000: // NOP
175                     begin
176                        // do nothing
177                     end
178
179                  4'b0001: // ADD
180                     begin
181                        LatchA      = 1'b1;
182                        EnableALU   = 1'b1;
183                     end
184
185                  4'b0010: // SUB
186                     begin
187                        LatchA      = 1'b1;
188                        EnableALU   = 1'b1;
189                        AddSub      = 1'b1;
190                     end
191
192                  4'b0011: // OUT
193                     begin
194                        // do nothing
195                     end
196
197                  4'b0100: // IN
198                     begin
199                        // do nothing
200                     end
201
202                  4'b0101: // LDA
203                     begin
204                        // do nothing
205                     end
206
207               endcase
208
209            end
210
211      endcase
```

FSM – RTL Viewer





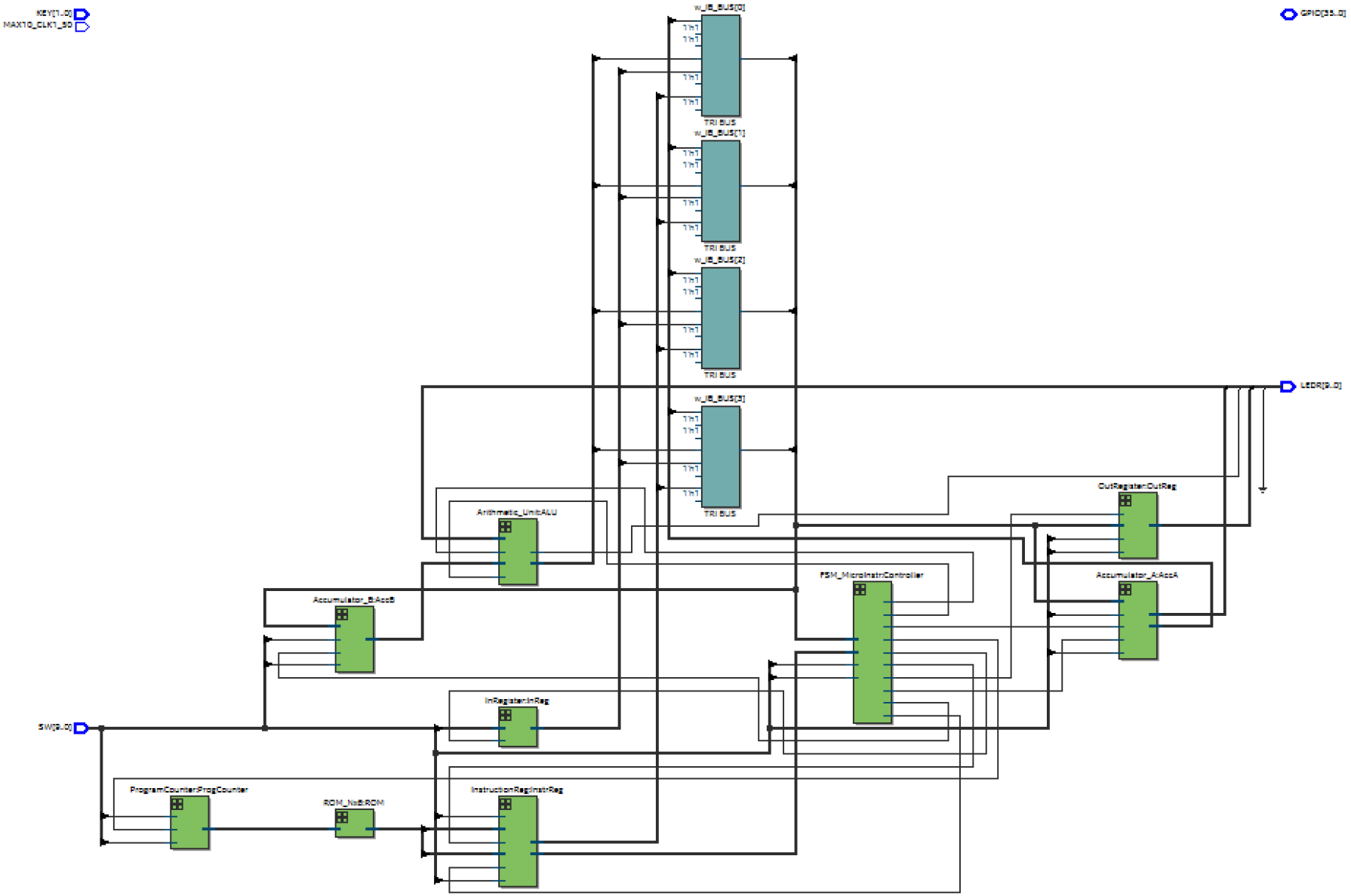
FSM – Test

```
// Microinstructions (FSM)
FSM_MicroInstr Controller (
    .clk(w_clock),
    .reset(w_reset),
    .IB_BUS(w_IB_BUS),
    .LatchA(w_LatchA),
    .EnableA(w_EnableA),
    .LatchB(w_LatchB),
    .EnableALU(w_EnableALU),
    .AddSub(w_AddSub),
    .EnableIN(w_EnableIN),
    .EnableOut(w_EnableOut),
    .LoadInstr(w_LoadInstr),
    .EnableInstr(w_EnableInstr),
    .ToInstr(w_ToInstr),
    .EnableCount(w_EnableCount)
);
```

Required Work

- **Finish the rest of the modules**
- **Include your code and the RTL Viewer**
- **Include verification and validation of your code**
- **Include a video showing the final microcontroller working**

Top Level


KEY[1..0]
MAX10_CLK1_50
w_IB_BUS[0]
w_IB_BUS[1]
w_IB_BUS[2]
w_IB_BUS[3]
TRI BUS
GPIO[35..0]
LEDR[9..0]
Arithmetic_UnitALU
OutRegisterOutReg
FSM_MicroInstrController
Accumulator_BAccB
Accumulator_AAccA
InRegisterInReg
SW[9..0]
ProgramCounterProgCounter
ROM_NotBROM
InstructionRegInstrReg

Flow Summary

Flow Summary

<<Filter>>

Flow Status	Successful - Tue Oct 28 02:23:34 2025
Quartus Prime Version	24.1std.0 Build 1077 03/04/2025 SC Lite Edition
Revision Name	main
Top-level Entity Name	main
Family	MAX 10
Device	10M50DAF484C7G
Timing Models	Final
Total logic elements	52 / 49,760 (< 1 %)
Total registers	27
Total pins	59 / 360 (16 %)
Total virtual pins	0
Total memory bits	0 / 1,677,312 (0 %)
Embedded Multiplier 9-bit elements	0 / 288 (0 %)
Total PLLs	0 / 4 (0 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0 / 2 (0 %)

Chip Planner

